

Digital Twin Architecture

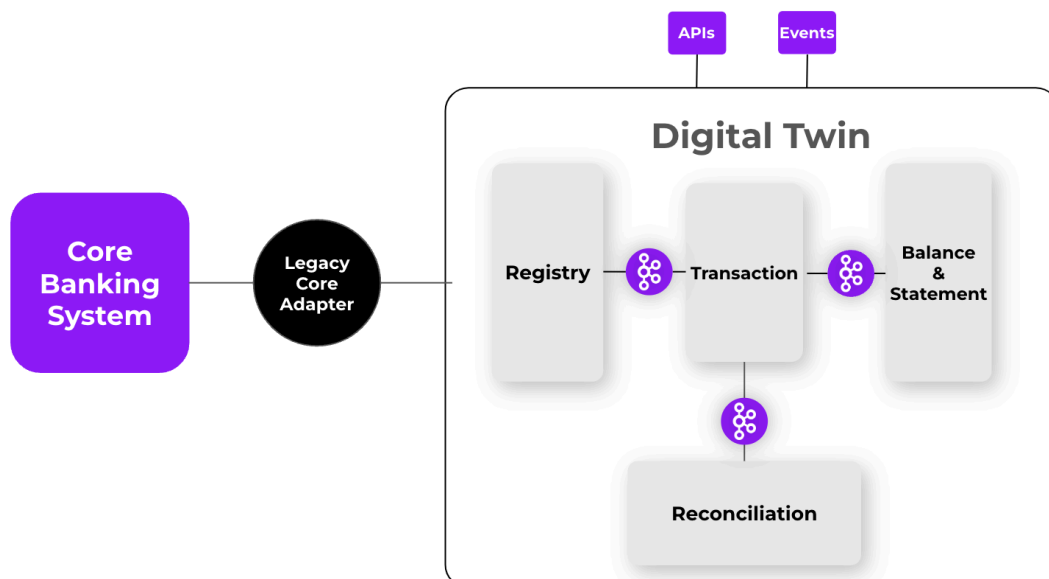
Digital Twin is built using a cloud-native Java microservices architecture. The platform consists of four core services:

- **Registry** – Maintains account, customer, and reference data.
- **Transaction Service** – Authorizes and executes financial operations, maintains balances, and orchestrates transactions across one or more accounts.
- **Balance & Statements** – Optimized for balance inquiries and transaction history.
- **Reconciliation** – Verifies data consistency between Digital Twin services and external systems, such as the core banking platform.

Each service maintains its own database, allowing services to scale independently while isolating failures and workloads. For example, a high volume of balance inquiries does not affect the Transaction Service's ability to authorize payments.

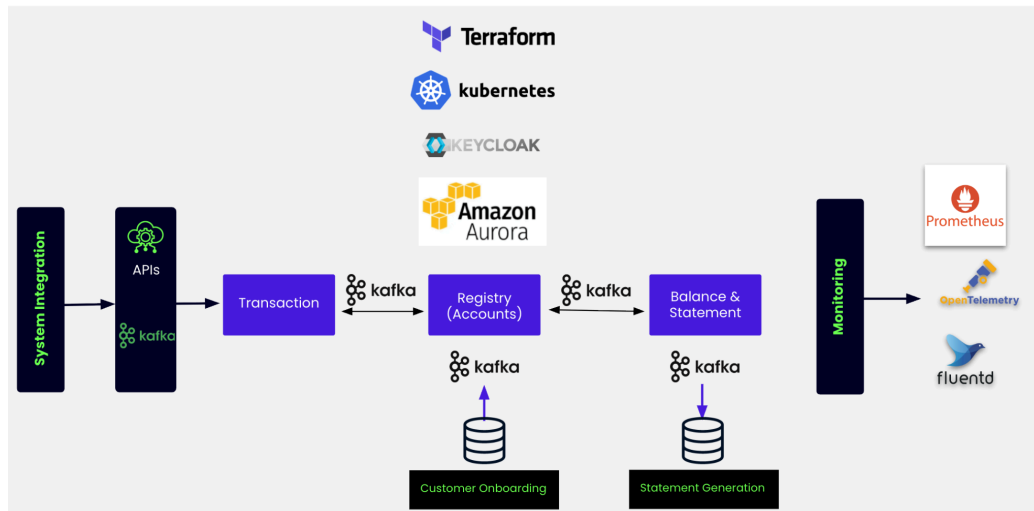
The microservices communicate asynchronously through Apache Kafka. When one service publishes an event, other services subscribe and react independently. For example, after the Transaction Service posts a payment, the Balance & Statements service consumes the event to update transaction history, while other services may independently consume the same event for reconciliation or additional processing.

This event-driven architecture provides independent scalability, fault isolation, and resilient operation while supporting high-throughput, real-time transaction processing.



Digital Twin Components

Digital Twin employs a microservice and container-based architecture, allowing for fast and flexible application creation and deployment. Additionally, the application is packaged into Helm Charts and runs through Kubernetes, making it easier to deploy and operate across different environments. Meanwhile, Kubernetes clusters are created and managed using Terraform technology.



- **Terraform**

Used to help set up and manage the cloud systems needed to run Digital Twin. Matara can provide a script configuring a base test system. **This script is for reference builds only** with no direct support. Clients can make modifications to the script to suit their planned environments such as the ability to configure AWS region, number and type of EC2 instances, database version, and other parameters.

- **Kubernetes (EKS)**

Allows Digital Twin to run in a way that's easy to manage, flexible and always available in the cloud.

- Groups containers into pods which can be scaled up during surges and self healed (restart/replace) as required
- Helps with load balancing and resource utilization
- Provides stability and ensures high availability
- Can use EC2 spot instances for cost savings vs On Demand but comes with the inherent risk of interruption
- Usage of other containerization applications, such as Docker, is possible but should be reviewed and confirmed by Matara

- **Keycloak**

Manages secure user authentication and access control ensuring only authorized users

and systems can interact with its APIs and resources. Matera provides the default resource roles that can be assigned as needed by the client. Clients may be added via a script or through the Account Management Console.

- **RDS Aurora**

Proprietary relational database service offered by AWS for high performance and availability. Compatible with MySQL (preferred), Oracle (licensed only) and SQL Server (licensed only).

- **MSK (Apache Kafka)**

An AWS managed service that simplifies the operation of Apache Kafka clusters for streaming data applications. Includes event streaming for transaction and balance/statement reporting. Key benefit is the handling of provisioning, configuration, and maintenance of Kafka clusters automatically. It allows automatic or manual scaling of partitions and brokers, ensuring consistent performance during traffic spikes. Usage of other Apache Kafka, such as Confluent, is possible but should be reviewed and confirmed by Matera.

Additional open-source applications to support Digital Twin

Digital Twin's design of metrics and logging pair well with the following monitoring applications. These are suggested applications but not included as part of the deployment of the solution.

- **Prometheus** is tool that collects, stores and analyzes metrics to monitor Digital Twin's performance
- **OpenTelemetry** provides a single set of APIs, libraries, agents, and collector services to capture distributed traces and metrics from your application. Export telemetry data to any observability backend such as Prometheus, Jaeger, commercial vendors, or your own solution.
- **Fluentd** lets you unify the data collection and consumption for a better use and understanding of data. It Allows developers and data analysts to utilize many types of logs as they are generated.

APIs and Event Streaming

Digital Twin supports both synchronous REST APIs and asynchronous event streaming using Apache Kafka.

- **REST APIs** are used for direct request/response interactions, such as transaction requests, balance inquiries, and account management.

- **Apache Kafka** provides the platform's event streaming backbone, enabling high-throughput, asynchronous communication between Digital Twin services and external systems.

Kafka supports two messaging patterns:

- **Events**, which notify interested systems that an operation has occurred.
- **Commands**, which request that Digital Twin perform a financial operation and return the result asynchronously.

Together, REST APIs and Kafka provide flexible integration options, allowing Digital Twin to support both synchronous user interactions and scalable event-driven processing.

See *Integration* for details on Inbound Events, Commands, Published Events, REST APIs, message flows, and implementation guidance.